Real Example Program

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

int main() {
    char buf[50];
    fgets(buf, sizeof(buf), stdin);

    int len = strlen(buf);

    // Case 1: numeric input
    if (buf[0] >= '0' && buf[0] <= '9') {
        int n = atoi(buf);
        printf("Number squared: %d\n", n*n);
    }

    // Case 2: very long input
    else if (len > 40) {
        printf("Long input detected (%d chars)\n", len);
    }

    // Case 3: contains %
    else if (strchr(buf, '%')) {
        fprintf(stderr, "Format string detected: %s\n", buf);
    }

    // Case 4: contains NULL-like pattern
    else if (buf[0] == '\0') {
        fprintf(stderr, "Empty input\n");
    }

    // Default case
    else {
        printf("Normal input: %s\n", buf);
    }

    return 0;
}
```

Crash-related features

1. `crash_count = 0`

This program never crashes

- no invalid memory access
- no segmentation fault

Even `"A"*50000` → safe (buffer limited by `fgets`)

2. `crash_rate = 0.0`

Same reason → all runs succeed

Memory bug features (all 0)

- heap_overflow
- stack_overflow
- use_after_free
- etc.

Why?

- buffer is fixed (`char buf[50]`)
- `fgets` prevents overflow

So fuzzing cannot trigger memory bug

Undefined behavior features

No division, no pointer misuse
So all = 0

Timeout

12. `timeouts = 0`

No loops → finishes instantly

Execution time

13. `avg_exec_time ≈ 0.07`

Depends on input length

Example:

"A" → very fast
"A"*50000 → slightly slower (processing length)

Exit behavior

14. `unique_exit_codes = 1`

Every path ends with:

return 0;

Even though logic differs → exit same

stderr length

15. `stderr_len ≈ 183`

Only some inputs print to stderr:

"%x%x" → Format string detected

Others print to stdout

Average becomes ~183

success_rate = 1.0

Every run returns 0 → success

behavioral features

17. `path_variability = 50`

Why high?

Because program has **multiple branches**:

if numeric → path 1
else if long → path 2
else if % → path 3
else default → path 4

Different inputs → different branches

our hash detects different stderr/output
→ counts as different “paths”

18. `unique_outputs = 50`

Example inputs:

"123" → Number squared: 15129
"AAAAA" → Normal input
"%x%x" → Format string detected
"A"*50 → Long input detected

All produce different outputs

So unique outputs = high

19. `time_variance = 0.03`

Why?

short input → fast
long input → slower (strlen, checks)
numeric → extra computation

Execution time varies → variance increases

20. `input_sensitivity = 49`

Almost every input changes behavior

Example:

"A" → Normal input
"123" → Number squared
"%x" → Format error
"A"*50 → Long input

Output changes every time → high sensitivity

21. `max_stderr_len = 184`

Longest stderr case:

Format string detected: %x%x%x%x...

This produces largest message

captured as max

Feature type	What it tells
Crash	Does it break?
Exit code	Does it end differently?
Output	What does it say?
Path variability	How many behaviors exist?
Sensitivity	Does input matter?
Time variance	Does complexity change?